

Aprendizado Profundo

Normalizing Flows

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Visão Geral

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Overview

- 1. Modelos Geradores – Revisão**
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Modelos Geradores

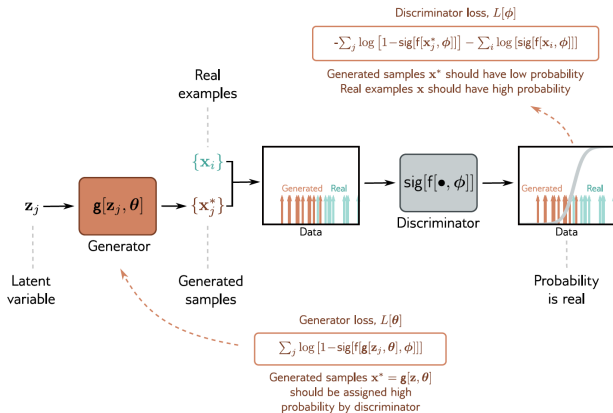
- O objetivo de um **modelo gerador** é aprender uma **distribuição de probabilidade** sobre uma variável aleatória $\mathbf{X}$ usando um conjunto de dados observados $\{x^{(i)}\}_{i=1}^N$.
- A densidade $p_X(x)$ é parametrizada por θ .
- Dois usos fundamentais de um modelo gerador:
 - **Amostrar**: obter novos exemplos a partir do modelo.
 - **Avaliar**: calcular $p_X(x)$ para uma instância dada.

Observação

Diferentes famílias de modelos geradores são fortes em um ou outro desses aspectos; poucos fazem os dois bem.

Modelos Geradores – Voltando aos GANs

- GANs:
 - **Amostrar** p_X : conseguimos (passagem pelo gerador).
 - **Avaliar** p_X : inviável, não há densidade explícita.
- Isso motiva a busca por modelos que também permitam avaliar p_X .

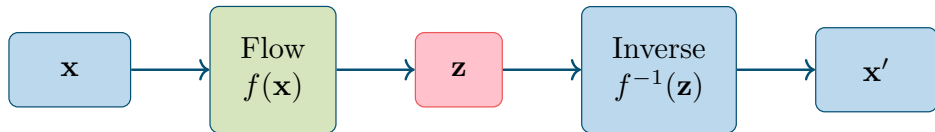


Overview

1. Modelos Geradores – Revisão
- 2. Visão Geral de Normalizing Flows**
3. Mudança de Variável
4. Loss
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Normalizing Flows¹

- *Normalizing Flows* são modelos geradores probabilísticos que permitem **tanto amostrar quanto avaliar** $p_X(x)$.
- Consistem em **transformar** uma distribuição tratável base (e.g. $z \sim \mathcal{N}(0, 1)$) em uma outra distribuição usando **funções inversíveis**.



¹Ivan Kobyzev, Simon J. D. Prince e Marcus A. Brubaker. “Normalizing Flows: An Introduction and Review of Current Methods”. Em: IEEE Transactions on Pattern Analysis and Machine Intelligence 43.11 (2020), pp. 3964–3979.

Normalizing Flows – Exemplo Glow²

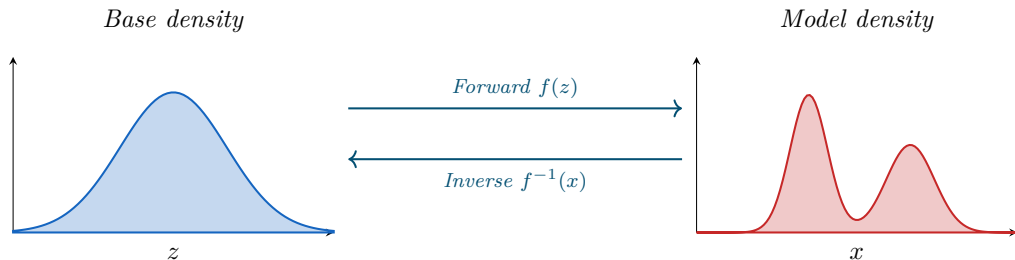


Faces sintetizadas pelo *Glow*, um *Normalizing Flow* com convoluções invertíveis 1×1 .

²Durk P. Kingma e Prafulla Dhariwal. “Glow: Generative Flow with Invertible 1×1 Convolutions”. Em: Advances in Neural Information Processing Systems 31 (2018).

Normalizing Flows – Transformação 1D

Gaussiana base $\rightarrow$ distribuição bimodal via *flow*:



- *Forward*: leva da distribuição base $p(z)$ para $p(x)$ (amostragem).
- *Inverse*: faz o caminho oposto (avaliação via mudança de variável).

Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
- 3. Mudança de Variável**
4. Loss
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Mudança de Variável – Caso discreto

Mudança de variável no caso discreto é **trivial**. Considere o lançamento de um dado:

$$p_X(x = 1) = \frac{1}{6}$$

- Aplicando uma função inversível, por exemplo $f(x) = x^2$, o espaço amostral do experimento muda:

$$\{1, 2, 3, 4, 5, 6\} \longrightarrow \{1, 4, 9, 16, 25, 36\}$$

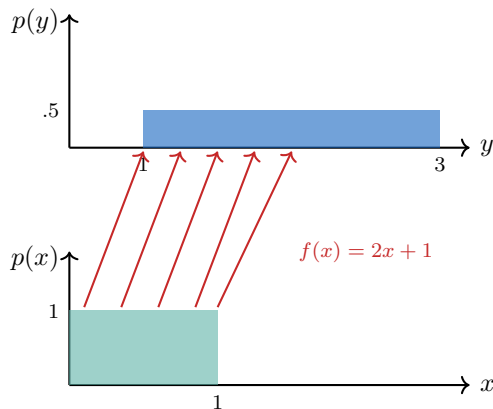
- Para calcularmos $p_Z(z = 9)$, basta perceber que $z = 9 \equiv x = 3$: **as probabilidades são iguais**.

Caso discreto

No discreto, cada evento mantém sua massa de probabilidade: só mudam os rótulos.

Mudança de Variável – Caso contínuo

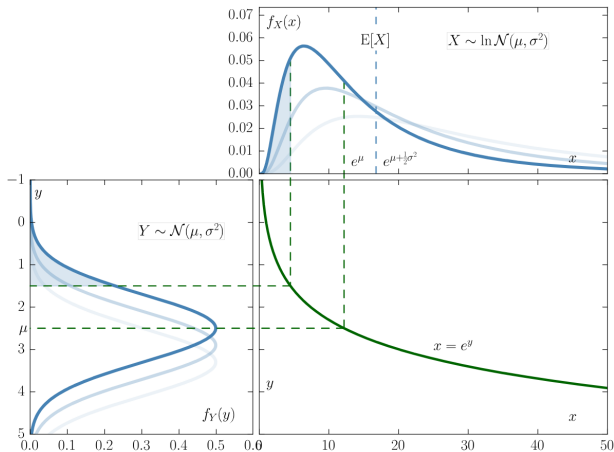
- No contínuo, a noção fundamental passa a ser **intervalo de probabilidade**, não evento pontual.
- Ao aplicar uma função **inversível e derivável**, a densidade sofre uma **deformação**.
- A massa em um volume pode ser achatada ou alongada ao passar pela transformação.
- A altura da densidade é **inversamente proporcional** ao alargamento: área total sempre = 1.



Uniforme $p(x)$ transformada em $p(y)$ mais larga e menos alta por $y = 2x + 1$.

Mudança de Variável – Exemplo visual

- Repare as duas distribuições ao lado.
- Um determinado volume de uma pode ser **achatado** ou **alongado** ao efetuar a transformação.
- A distribuição gaussiana (abaixo) é transformada em uma log-normal (acima) por $x = e^y$.



Mudança de Variável – Equação base

A conservação de probabilidade exige que infinitesimais de probabilidade se preservem:

$$p(x) dx = p(z) dz$$

- Isso é a lei fundamental da mudança de variável em densidades.
- Intuitivamente: a área sob a curva em um intervalo transformado deve ser igual à área no intervalo original.

Mudança de Variável – Fórmula explícita

Reorganizando, expressamos $p(x)$ em função de $p(z)$:

$$p(x) dx = p(z) dz \implies p(x) = p(z) \left| \frac{dz}{dx} \right|$$

Por que o módulo?

Garante que:

- o resultado seja **positivo** (densidades são não negativas);
- não precisamos nos preocupar com direções de integração/derivação.

Mudança de Variável – Exemplo passo a passo

Setup: $X \sim \mathcal{N}(0, 1)$ e $Z = \exp(X)$. Calcular $p_Z(z)$.

Passos

1. Densidade de X : $p_X(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$.
2. Como $z = \exp(x)$, a inversa é $x = \ln(z)$.
3. $P(Z \leq z) = P(X \leq \ln z)$, logo $\text{CDF}_Z(z) = \text{CDF}_X(\ln z)$.
4. Derivando: $p_Z(z) = p_X(\ln z) \frac{\partial(\ln z)}{\partial z}$.
5. Substituindo: $p_Z(z) = \frac{1}{z\sqrt{2\pi}} \exp\left(-\frac{(\ln z)^2}{2}\right)$.
6. Essa é a densidade **log-normal**.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
- 4. Loss**
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Loss – *Maximum Likelihood*

Para treinar, seguimos o *framework* de maximizar a **verossimilhança** dos dados:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[\prod_{i=1}^N P(x^{(i)} \mid \theta) \right]$$

- O produto vem da suposição de independência das amostras.
- Usualmente trabalhamos com a log-verossimilhança (soma, não produto) e a minimizamos com sinal trocado.

Loss – NLL passo a passo (1/2)

Passo 1: trocamos produto por soma negativa (NLL):

$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[\prod_i P(x^{(i)} \mid \theta) \right]$$

Produtos são instáveis numericamente e gradientes ficam feios; vamos logaritmar.

Loss – NLL passo a passo (1/2)

Passo 1: trocamos produto por soma negativa (NLL):

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} \left[\prod_i P(x^{(i)} \mid \theta) \right] \\ &= \operatorname{argmin}_{\theta} \left[\sum_{i=1}^N -\log P(x^{(i)} \mid \theta) \right]\end{aligned}$$

Mudança de variável do *flow*

$$p_X(x) = p_Z(z) \left| \frac{\partial f(z, \theta)}{\partial z} \right|^{-1}$$

Vamos substituir $P(x^{(i)} \mid \theta)$ na expressão acima.

Loss – NLL passo a passo (2/2)

Passo 2: substituir p_X pela mudança de variável:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \left[\sum_{i=1}^N -\log \left(P(z^{(i)}) \left| \frac{\partial f(z^{(i)}, \theta)}{\partial z^{(i)}} \right|^{-1} \right) \right]$$

Loss – NLL passo a passo (2/2)

Passo 2: substituir p_X pela mudança de variável:

$$\begin{aligned}\hat{\theta} &= \operatorname{argmin}_{\theta} \left[\sum_{i=1}^N -\log \left(P(z^{(i)}) \left| \frac{\partial f(z^{(i)}, \theta)}{\partial z^{(i)}} \right|^{-1} \right) \right] \\ &= \operatorname{argmin}_{\theta} \left[\sum_{i=1}^N \left[\log \left| \frac{\partial f(z^{(i)}, \theta)}{\partial z^{(i)}} \right| - \log P(z^{(i)}) \right] \right]\end{aligned}$$

Dois termos, dois papéis

- $\log |\det \nabla f|$: depende de θ via a rede. Penaliza contrair demais o volume.
- $-\log P(z)$: depende apenas da distribuição base (gaussiana). Tratável em forma fechada.

Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
- 5. O que são Flows**
6. Coupling Flows
7. Discreto vs. Contínuo
8. Panorama Geral

Flows

Um *flow* é uma função $f(\mathbf{x})$ com quatro propriedades essenciais:

- **Paramétrica:** queremos definir parâmetros aprendíveis.
- **Inversível:** queremos calcular o *flow* nos dois sentidos (amostragem e avaliação).
- **Diferenciável:** queremos otimizar via *backprop*.
- **Jacobiana tratável:** conseguimos computar a inversa e o determinante da Jacobiana de **maneira eficiente**.

Desafio de engenharia

Satisfazer os quatro requisitos simultaneamente é o que define as arquiteturas de *flows*.

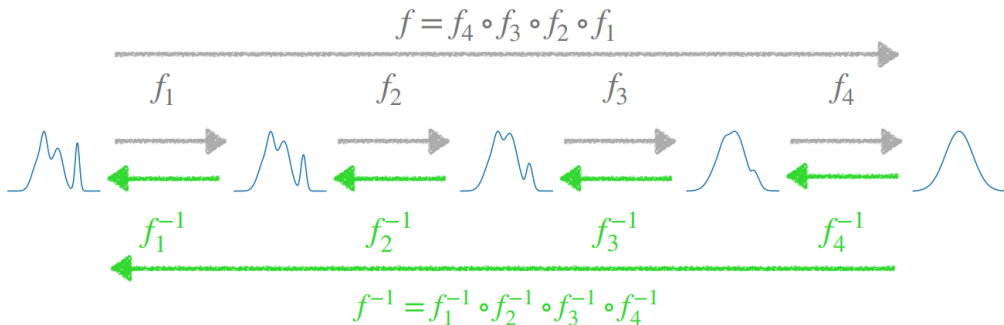
Flows – Composição

A classe de funções diferenciáveis e inversíveis é **fechada em composição**:

$$f = f_k \circ f_{k-1} \circ \cdots \circ f_2 \circ f_1$$

- Isso nos possibilita **projetar camadas** que computam um *flow* e empilhá-las em redes profundas.
- A expressividade vem da composição: cada camada pode ser simples desde que a composição seja rica.

Flows – Caminho direto e inverso



- Caminho direto: $f = f_4 \circ f_3 \circ f_2 \circ f_1$ transforma a distribuição base em $p(x)$ (cinza).
- Caminho inverso: $f^{-1} = f_1^{-1} \circ f_2^{-1} \circ f_3^{-1} \circ f_4^{-1}$ faz o retorno (verde).

Flows – Jacobiana da composição

Pela regra da cadeia, o determinante da Jacobiana de uma composição é o **produto dos determinantes**:

$$\left| \frac{\partial f[z, \boldsymbol{\theta}]}{\partial z} \right| = \left| \frac{\partial f_k[f_{k-1}, \boldsymbol{\theta}_k]}{\partial f_{k-1}} \right| \cdot \left| \frac{\partial f_{k-1}[f_{k-2}, \boldsymbol{\theta}_{k-1}]}{\partial f_{k-2}} \right| \dots \left| \frac{\partial f_1[z, \boldsymbol{\theta}_1]}{\partial z} \right|$$

- Por isso, basta que cada camada tenha Jacobiana fácil de calcular para que a rede inteira também tenha.
- Esse é o princípio que guia o design de arquiteturas de *flows*.

Linear Flow

Uma transformação **linear** pode ser um *Linear Flow* se a matriz admitir inversa:

$$f(\mathbf{x}) = \boldsymbol{\theta}\mathbf{x} + \mathbf{b} \quad f^{-1}(\mathbf{z}) = \boldsymbol{\theta}^{-1}(\mathbf{z} - \mathbf{b})$$

Problemas

- **Pouco expressiva:** composição de transformações lineares continua linear.
- Determinante e inversa de matriz densa: custo $\mathcal{O}(d^3)$.

Linear Flow – Custo por estrutura

- Impondo estrutura na matriz θ , reduzimos drasticamente o custo.
- Diagonal é barata mas muito restritiva.
- LU factorized e 1x1 convolution (Glow) são os melhores compromissos na prática.

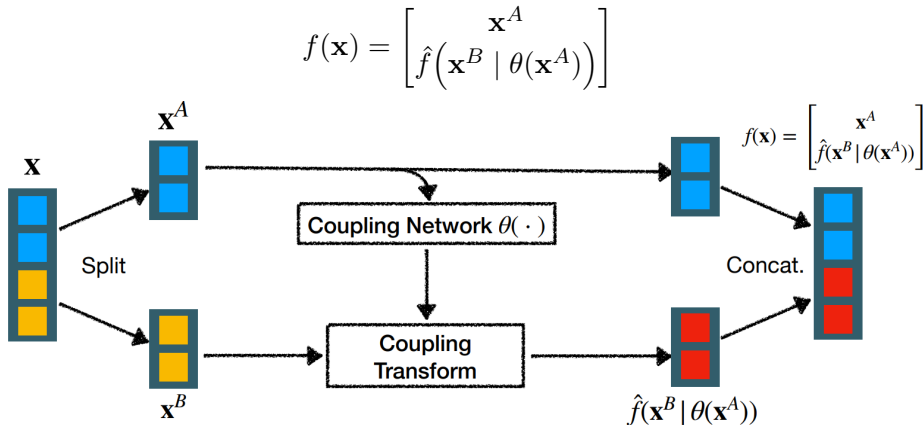
	Inverse	Determinant
Full	$O(d^3)$	$O(d^3)$
Diagonal	$O(d)$	$O(d)$
Triangular	$O(d^2)$	$O(d)$
Block Diagonal	$O(c^3 d)$	$O(c^3 d)$
LU Factorized [Kingma and Dhariwal 2018]	$O(d^2)$	$O(d)$
Spatial Convolution [Hoogeboom et al 2019; Karami et al., 2019]	$O(d \log d)$	$O(d)$
1x1 Convolution [Kingma and Dhariwal 2018]	$O(c^3 + c^2 d)$	$O(c^3)$

Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
5. O que são Flows
- 6. Coupling Flows**
7. Discreto vs. Contínuo
8. Panorama Geral

Coupling Flows³

Dividir a entrada em duas metades e transformar apenas uma delas, condicionando na outra:

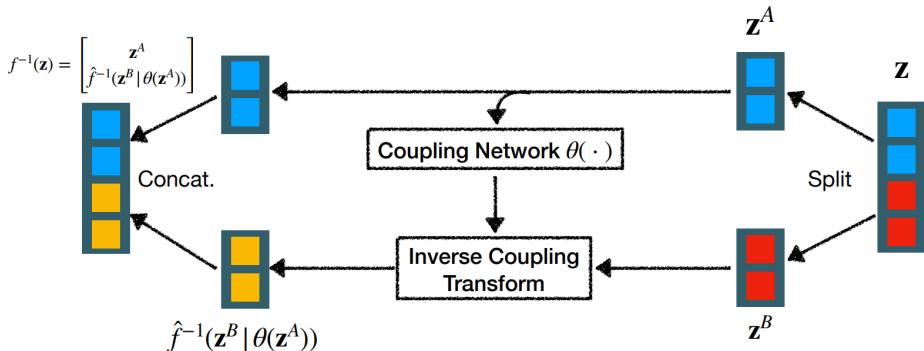


³Laurent Dinh, Jascha Sohl-Dickstein e Samy Bengio. “Density Estimation using Real NVP”. Em: International Conference on Learning Representations. 2017.

Coupling Flows – Inversa

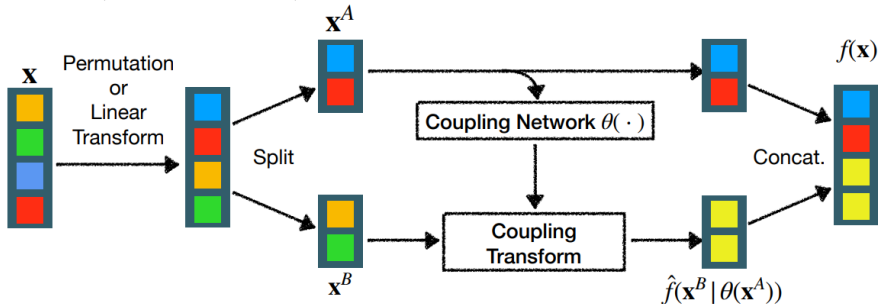
Como $\mathbf{x}^A$ é copiado direto, temos $\mathbf{z}^A = \mathbf{x}^A$. Para inverter basta aplicar $\hat{f}^{-1}$ em $\mathbf{z}^B$ usando a mesma *coupling network* que produziu os parâmetros:

$$f^{-1}(\mathbf{z}) = \begin{bmatrix} \mathbf{z}^A \\ \hat{f}^{-1}(\mathbf{z}^B | \theta(\mathbf{z}^A)) \end{bmatrix}$$



Coupling Flows – Combinando com permutações

Um único *coupling layer* só transforma metade das dimensões. **Alternando permutações** (ou *linear flows*) entre as camadas, todas as dimensões são afetadas:



Cada camada: permutação $\rightarrow$ split $\rightarrow$ coupling transform $\rightarrow$ concat.

Coupling Flows – Jacobiana por blocos

Por que essa arquitetura? Porque a Jacobiana é **triangular por blocos**:

$$Df(\mathbf{x}) = \begin{bmatrix} \frac{\partial \mathbf{x}^A}{\partial \mathbf{x}^A} & \frac{\partial \mathbf{x}^A}{\partial \mathbf{x}^B} \\ \frac{\partial \hat{f}}{\partial \mathbf{x}^A} & \frac{\partial \hat{f}}{\partial \mathbf{x}^B} \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \frac{\partial \hat{f}}{\partial \mathbf{x}^A} & \frac{\partial \hat{f}}{\partial \mathbf{x}^B} \end{bmatrix}$$

- $\mathbf{x}^A \rightarrow \mathbf{x}^A$: identidade (bloco superior-esquerdo = $\mathbf{I}$).
- $\mathbf{x}^A \rightarrow \mathbf{x}^A$ via B : zero (pois $\mathbf{x}^A$ é só copiado).

Coupling Flows – Exemplo concreto em 4D

Seja $\mathbf{x} = (x_1, x_2, x_3, x_4)$ com $\mathbf{x}^A = (x_1, x_2)$ e $\hat{f}(\mathbf{x}^B) = (g_1(x_1, x_2) \cdot x_3, g_2(x_1, x_2) \cdot x_4)$:

$$Df = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{\partial g_1}{\partial x_1} x_3 & \frac{\partial g_1}{\partial x_2} x_3 & g_1 & 0 \\ \frac{\partial g_2}{\partial x_1} x_4 & \frac{\partial g_2}{\partial x_2} x_4 & 0 & g_2 \end{bmatrix}$$

Determinante via triangularidade

$$\det Df = 1 \cdot 1 \cdot g_1 \cdot g_2 = g_1 \cdot g_2$$

Basta o **produto dos elementos diagonais do bloco inferior-direito**. Os termos da linha inferior à esquerda **não importam** para o determinante.

Coupling Flows – Determinante geral

Generalizando, o determinante da Jacobiana é:

$$|\det Df| = \left| \det \frac{\partial \hat{f}(\mathbf{x}^B | \theta(\mathbf{x}^A))}{\partial \mathbf{x}^B} \right|$$

- Só depende do bloco inferior-direito.
- Se $\hat{f}$ for aplicada **elemento a elemento**, esse bloco é **diagonal** e o determinante é trivial.
- Consequência: todos os modelos modernos de *flow* para imagens (*RealNVP*, *Glow*, *Flow++*) usam alguma variante dessa ideia.

Affine Coupling Flows⁴ – Forward

A escolha mais comum de $\hat{f}$ é uma **transformação afim** elemento a elemento:

Forward pass

$$\mathbf{y}^B = \mathbf{x}^B \odot \exp(\mathbf{s}(\mathbf{x}^A)) + \mathbf{t}(\mathbf{x}^A), \quad \mathbf{y}^A = \mathbf{x}^A.$$

- A coupling network $\theta(\mathbf{x}^A)$ produz **dois vetores**: $\mathbf{s}$ (log-escala) e $\mathbf{t}$ (translação).
- $\exp(\cdot)$ garante escala positiva e permite expressar alongamento ($s > 0$) e compressão ($s < 0$).
- Jacobiana **diagonal**: $\det = \prod_i \exp(s_i)$, logo $\log |\det| = \sum_i s_i$ (barato).

⁴Laurent Dinh, Jascha Sohl-Dickstein e Samy Bengio. “Density Estimation using Real NVP”. Em: International Conference on Learning Representations. 2017.

Affine Coupling Flows – Inverse

A inversa é **quase de graça**: note que a coupling network usa apenas $\mathbf{x}^A$, que é copiado para $\mathbf{y}^A$. Então podemos **reaplicá-la** na direção inversa:

Inverse pass

1. $\mathbf{x}^A = \mathbf{y}^A$ (cópia direta).
2. Calcule $\mathbf{s} = \mathbf{s}(\mathbf{x}^A)$, $\mathbf{t} = \mathbf{t}(\mathbf{x}^A)$ usando a mesma rede.
3. $\mathbf{x}^B = (\mathbf{y}^B - \mathbf{t}) \odot \exp(-\mathbf{s})$.

Crucial

A *coupling network* pode ser **qualquer** rede neural (não precisa ser inversível!) – porque ela nunca é invertida, só reavaliada no lado correto.

Exemplo Numérico: Forward + Inverse

Pense em $\mathbf{x}$ como um vetor de features 2D de um ponto do dataset; $\mathbf{y}$ é o mesmo ponto após uma camada de *flow*, mais próximo da base $\mathcal{N}(0, \mathbf{I})$. s, t são saídas de uma pequena rede neural (a coupling network) que recebe $\mathbf{x}^A$.

Entrada $\mathbf{x} = (1, 2)$, split $\mathbf{x}^A = 1$, $\mathbf{x}^B = 2$. A *coupling network* produz:

$$s(\mathbf{x}^A) = 0,3, \quad t(\mathbf{x}^A) = -0,5.$$

Forward

$$\mathbf{y}^A = 1$$

$$\begin{aligned}\mathbf{y}^B &= 2 \cdot e^{0,3} - 0,5 \\ &\approx 2,70 - 0,5 = 2,20\end{aligned}$$

$$\log |\det J| = s = 0,3$$

Inverse

$$\mathbf{x}^A = 1$$

$$\begin{aligned}\mathbf{x}^B &= (2,20 + 0,5) \cdot e^{-0,3} \\ &\approx 2,70 \cdot 0,741 = 2,00 \checkmark\end{aligned}$$

A *coupling network* foi reavaliada na inversa mas nunca invertida; a inversa é fechada e custa o mesmo que a forward.

Affine Coupling Flows – PyTorch

```
1 class AffineCoupling(nn.Module):
2     def __init__(self, d, hidden=128):
3         super().__init__()
4         self.half = d // 2
5         self.net = nn.Sequential(
6             nn.Linear(self.half, hidden), nn.ReLU(),
7             nn.Linear(hidden, 2 * self.half),
8         )
9
10    def forward(self, x):
11        x_a, x_b = x[:, :self.half], x[:, self.half:]
12        s, t = self.net(x_a).chunk(2, dim=-1)
13        s = torch.tanh(s)
14        y_b = x_b * torch.exp(s) + t
15        log_det = s.sum(dim=-1)
16        return torch.cat([x_a, y_b], dim=-1), log_det
17
18    # inverse: swap + / - and exp(s) / exp(-s)
```

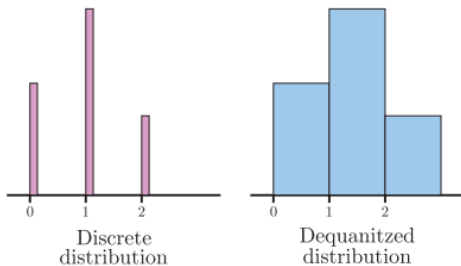
Ideias-chave: split em A/B ; rede prediz (s, t) a partir de A ; $\tanh$ em s estabiliza; $\log_det$ acumulado é a soma dos s .

Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
5. O que são Flows
6. Coupling Flows
- 7. Discreto vs. Contínuo**
8. Panorama Geral

Discreto vs. Contínuo – *Dequantization*

- Como **imagens são quantizadas** (pixels inteiros 0–255), podemos ter problemas de singularidades ao ajustar uma densidade contínua.
- Solução: adicionar **ruído**, usualmente uniforme em $[0, 1)$, à distribuição modelada.
- Isso transforma picos discretos em uma densidade contínua passível de modelagem.



Overview

1. Modelos Geradores – Revisão
2. Visão Geral de Normalizing Flows
3. Mudança de Variável
4. Loss
5. O que são Flows
6. Coupling Flows
7. Discreto vs. Contínuo
- 8. Panorama Geral**

Normalizing Flows – Aplicações

NFs são fracos em geração de imagens mas fortes onde a densidade tratável vale mais que a qualidade visual:

Detecção de anomalias

Treinamos em dados normais; amostras anômalas terão $\log p_X(x)$ baixo. Usado em monitoramento industrial e segurança de redes.

Inferência variacional

NFs substituem a distribuição $q(z | x)$ do VAE por uma distribuição **expressiva**, melhorando o ELBO.

Física e química computacional

Boltzmann Generators amostram conformações moleculares em equilíbrio, precisa densidade exata.

Amostragem em 3D e áudio

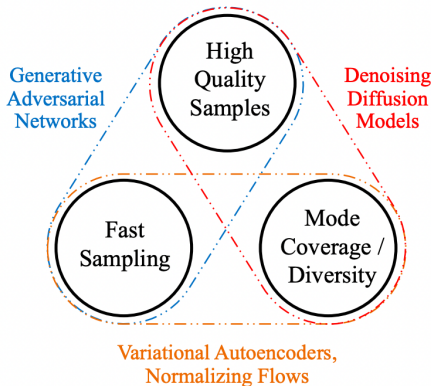
Glow, *WaveGlow*: geração de áudio e vídeo estritamente probabilística.

Onde NFs ficam no trilema?

- **Amostragem rápida:** ✓ (1 forward).
- **Cobertura / densidade:** ✓ (treino por MLE cobre toda a distribuição).
- **Qualidade visual:** × (pior que GANs e *diffusion*).

Posicionamento

NFs ocupam o canto velocidade + cobertura do trilema. Para qualidade visual máxima, os modelos difusores (próxima aula) vencem à custa da amostragem lenta.



- *Normalizing Flows* são modelos geradores capazes de **amostragem** e **avaliação da probabilidade**.
- São construídos com base em uma série de **transformações invertíveis**, com Jacobianas tratáveis.
- Ainda não possuem a mesma qualidade visual de GANs ou modelos difusores.
- No trilema qualidade/velocidade/cobertura, ocupam o vértice velocidade + cobertura (com qualidade inferior).

Quando usar NF

Quando precisar de **densidade tratável**: detecção de anomalias, compressão, inferência variacional, física computacional.

- *Normalizing Flows*: transformações **inversíveis e diferenciáveis** de uma distribuição base em uma modelada.
- Treinados por **máxima verossimilhança**, usando a fórmula de mudança de variável com o log-determinante da Jacobiana.
- *Linear flows* são limitados; estruturas como LU, 1×1 convolução reduzem custo.
- *Coupling flows* (RealNVP, Glow) dão expressividade com Jacobiana triangular.
- *Dequantization* é necessária para dados discretos (imagens).

Leituras Recomendadas

- Capítulo 16: Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Ivan Kobyzev, Simon J. D. Prince e Marcus A. Brubaker. “Normalizing Flows: An Introduction and Review of Current Methods”. Em: IEEE Transactions on Pattern Analysis and Machine Intelligence 43.11 (2020), pp. 3964–3979
- Durk P. Kingma e Prafulla Dhariwal. “Glow: Generative Flow with Invertible 1x1 Convolutions”. Em: Advances in Neural Information Processing Systems 31 (2018)
- Laurent Dinh, Jascha Sohl-Dickstein e Samy Bengio. “Density Estimation using Real NVP”. Em: International Conference on Learning Representations. 2017

Referências I

- [1] Laurent Dinh, Jascha Sohl-Dickstein e Samy Bengio. “Density Estimation using Real NVP”. Em: International Conference on Learning Representations. 2017.
- [2] Durk P. Kingma e Prafulla Dhariwal. “Glow: Generative Flow with Invertible 1x1 Convolutions”. Em: Advances in Neural Information Processing Systems 31 (2018).
- [3] Ivan Kobyzev, Simon J. D. Prince e Marcus A. Brubaker. “Normalizing Flows: An Introduction and Review of Current Methods”. Em: IEEE Transactions on Pattern Analysis and Machine Intelligence 43.11 (2020), pp. 3964–3979.
- [4] Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023.